

AuditLane: A Reproducible Pre-Audit Cleanroom for the Arbitrum Audit Program

Dipankar Sarkar
doom2quake

August 2026

Abstract

A project that lands audit funding still hands the auditor a raw repository: a build that works on one machine and not another, a test suite that needs undocumented setup, and deployment scripts that point at stale addresses. Under a subsidised audit program the firm spends the first days of a fixed engagement reconstructing the build before a single line is reviewed, and that reconstruction is paid for out of a subsidy meant to buy review. We present AuditLane, a reproducible pre-audit cleanroom for the Arbitrum Audit Program [1]. AuditLane folds a target Arbitrum repository into a single content-addressed *manifest*: a pinned toolchain, the captured test counts, and every contract’s source hash and Arbitrum Sepolia deployed-bytecode hash checked against the locally built artifact. The fold is a pure function of the repository state, so a reviewer re-runs the documented command and obtains a byte-for-byte identical manifest id, and a deployed bytecode that does not match the built artifact fails the package closed with a fixed reason code. The same seven reason codes, in the same fixed order, are recomputed on-chain by a reference contract, so a whitelisted firm never has to trust the tool’s word for the verdict. This paper describes the design and the milestone-1 implementation: a dependency-free Python core and a byte-for-byte Solidity reference, verified by 24 Python tests and 13 Solidity tests. All work is Arbitrum Sepolia testnet only; no mainnet, no deployment, no users, and no audit (Section 5).

1 Problem

An audit is only as deep as the time that survives the setup. When a whitelisted firm opens a repository cold, it reconstructs a build environment (which compiler version, which optimizer settings, which library commits), works out how to run the test suite, and reverse-engineers which on-chain address corresponds to which contract, all before review begins. Three failure shapes recur. First, the build is *irreproducible*: an unpinned compiler range or an environment-dependent step means the artifact the firm builds is not the artifact that was deployed. Second, the evidence is *asserted, not carried*: the team states that tests pass and that a deployed address holds a given contract, but nothing in the handoff lets the firm confirm either without redoing the work. Third, the review is *not re-runnable*: after fixes, a re-audit starts the same archaeology over again, because the first pass captured nothing a machine can replay.

The cost lands on the subsidy. The Arbitrum Audit Program [1] allocates \$10M in ARB over twelve months to compensate a fixed whitelist of firms for third-party smart-contract audits on Arbitrum [7]. Every hour a firm spends reconstructing a build is subsidy that bought no findings. The gap is narrow and concrete: there is no standard, reproducible, evidence-carrying package a project produces *before* the firm starts, so the firm opens a build that already works, a test suite that already ran with its counts captured, and a verdict anyone can recompute.

2 Why Arbitrum

AuditLane is Arbitrum-specific by construction, and not because of a superficial integration. Its manifest is content-addressed, and the reproducibility verdict is a pure function whose seven reason codes and fixed evaluation order are mirrored byte-for-byte in a reference contract, `BuildRegistry.sol`. A project commits a manifest id on Arbitrum Sepolia; anyone recomputes the same id from the published package and checks the two agree, and the commitment is *write-once* so a claim is immutable once made. The result is that “this build reproduced, here is the verdict” becomes a public, timestamped, on-chain claim on Arbitrum [2], not a line in a report the reader is asked to believe. The timing is the program itself: the Audit Program is live, funded for a fixed window, and routing rolling engagements now, so a tool that reduces setup waste is worth most while the pipeline is running.

3 Design

AuditLane is one pure core, one on-chain reference, and one agent wrapper.

3.1 The manifest: a pure fold

The load-bearing component (`auditlane/manifest.py`) takes a repository spec — the pinned toolchain, the source files committed by content hash, the captured test-suite result, and the on-chain deployments — and folds it into a canonical payload over which a single content-addressed `manifest_id` is taken. The fold performs no I/O, uses no clock, and touches no network: every value that varies is passed in, so the digest is a function of evidence alone [5]. Sources and contracts are sorted by a stable key before hashing, so their order in the repository cannot change the id, and the canonical field order is fixed and mirrored in Solidity. Leaf content is committed as SHA-256 [6] hashes, so the manifest never carries source bytes, only commitments to them. The consequence is the reproducibility property milestone 1 promises: two runs over the same repository state produce a byte-identical manifest and the same id, and a reviewer who clones the repository and re-runs the documented command reproduces the id and the test counts we publish.

3.2 The reproducibility check: fixed order, fail closed

A deterministic `check` returns the reason codes that apply, in a fixed order; an empty list is `REPRODUCIBLE`. The codes, in order, are: an unpinned toolchain (1), a build that captured no sources (2), a declared test suite that did not pass (3), a contract whose source is missing from the package (4), a deployed bytecode that does not match the built artifact (5), and a contract with no resolved on-chain deployment (6). The check is fail-closed: any unmet condition adds a code, and a green manifest carries none. Critically, the scanner does *not* synthesise a passing test suite: the counts come from a captured `auditlane.tests.json` written after the suite ran, so the manifest reports the counts that actually ran.

3.3 The on-chain reference: `BuildRegistry.sol`

`BuildRegistry` recomputes the same verdict on-chain from a package’s checkable shape, with the identical reason-code constants and the identical evaluation order, so the first-failing code returned on-chain matches the off-chain verdict. `manifestDigest` recomputes the id from the canonical payload with Keccak-256 [3], and `commit` writes a manifest id once, reverting on a re-commit, so a

package’s claim is a public, timestamped, immutable commitment. The two implementations share a common ABI: the same reason codes, the same order, the same canonical joining.

3.4 The Arbitrum chain seam

The on-chain claim AuditLane makes is: “the contract deployed at this Arbitrum Sepolia address is the contract in this repository.” To check that without trusting the tool, the manifest carries the hash of the deployed runtime bytecode next to the hash of the locally built artifact, and the check fails closed if they diverge. The seam (`auditlane/chain.py`) is one interface with two backends. Offline (the default, keyless) it returns the deployed bytecode hash from a deterministic in-process fixture, seeded from the sample’s own artifact bytes so the honest case reproduces and a tampered artifact fails closed. Live (gated on `AUDITLANE_RPC_URL`) it calls `eth_getCode` on Arbitrum Sepolia and refuses any chain id other than 421614. It is read-only either way and never needs a key.

3.5 The agent wrapper

The build runs as a recorded action wrapped in agent-core guardrails (`auditlane/agent.py`): the manifest write passes an `ActionLimiter`, so a runaway build loop is rate-limited like any other outbound action, and every step (scan, resolve, reproducibility check, manifest) is journalled in a `StateStore` so a UI can replay the activity log. The agent never emits a green manifest a failing check produced; that is the trust primitive.

4 Implementation and evaluation

The core is dependency-free Python: hashing, the canonical payload, and the check are a few hundred lines with no third-party runtime dependency, so “offline mode” can never degrade into an import error. The reference contract targets solc 0.8.24 and is self-contained. We evaluate by test, using Foundry [4] for the contract:

- **13 Solidity tests** (`forge test`, solc 0.8.24) on `BuildRegistry.sol`: every reason code, the fixed check order, the write-once commitment, and the Keccak digest.
- **24 Python tests** (`pytest`), keyless and offline: the reproducibility check for every code and its order, manifest determinism and content-addressing, the scanner reading a real fixture repository, and the agent recording a build with fail-closed guardrails.

The two suites are built to agree: the `test_check_*` cases are the exact analogues of the Solidity `test*Fails` cases, so a green `pytest` proves the off-chain core and the on-chain reference produce the same reason code, in the same order, for the same package. The sharpest case is the tampered deployment: a resolver returns the wrong deployed bytecode for the sample’s address, and the build yields `NOT_REPRODUCIBLE` with `BYTECODE_MISMATCH` (code 5), off-chain and on-chain alike.

5 Limitations

This is milestone 1, and the honest scope is narrow.

- **No mainnet**, and none planned under this grant. All Web3 targets are Arbitrum Sepolia testnet only; `BuildRegistry.sol` is not deployed anywhere yet (testnet deployment is milestone 2).

- **No live-chain test.** The live `eth_getCode` path is wired and gated on `AUDITLANE_RPC_URL`, but the test suite runs the offline fixture, not a real Arbitrum endpoint.
- **No users, no revenue, no audit, no partnership with the Arbitrum Foundation.** Milestone 3 is a request to be audited through the Audit Program, not a claim that we have been. The substitute for traction is tested code and a candid scope map, not usage numbers.

6 Conclusion

AuditLane makes the reproducibility of an Arbitrum project’s build a checkable artifact rather than an assertion: a content-addressed manifest a whitelisted firm opens without reconstruction, a fail-closed verdict with a fixed reason-code order, and a byte-for-byte on-chain reference that turns the verdict into a public, recomputable commitment on Arbitrum. Milestone 1 delivers the cleanroom core and the reference contract, tested. The durable contribution is a standard, reproducible, evidence-carrying handoff format that makes every subsidised audit in the program start from a better place.

References

- [1] Arbitrum Foundation. Arbitrum audit program. <https://arbitrum.foundation/>, 2026. Accessed 2026-08-24.
- [2] Arbitrum Foundation. Arbitrum documentation. <https://docs.arbitrum.io/>, 2026. Accessed 2026-08-24.
- [3] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. The Keccak reference, version 3.0. <https://keccak.team/files/Keccak-reference-3.0.pdf>, 2011.
- [4] Foundry contributors. Foundry: a portable and modular toolkit for Ethereum application development. <https://github.com/foundry-rs/foundry>, 2024.
- [5] Chris Lamb and Stefano Zacchiroli. Reproducible builds: Increasing the integrity of software supply chains. *IEEE Software*, vol. 39, no. 2, 2022. <https://doi.org/10.1109/MS.2021.3073045>.
- [6] National Institute of Standards and Technology. FIPS 180-4: Secure hash standard (SHS). <https://doi.org/10.6028/NIST.FIPS.180-4>, 2015.
- [7] Offchain Labs. Arbitrum: a suite of Ethereum scaling solutions. <https://arbitrum.io/>, 2026. Accessed 2026-08-24.